- **构造函数**
  - 说明：构造函数是专门用于创建对象的函数，如果一个函数使用 new 关键字调用，那么这个函数就是构造函数。

```
<script>
  // 定义函数
  function foo() {
    console.log('通过 new 也能调用函数...');
  }
  // 调用函数
  new foo();
</script>
```

  - 总结：
    - 使用 new 关键字调用函数的行为被称为实例化
    - 实例化构造函数时没有参数时可以省略 ()
    - 构造函数内部的 return 返回的值无效，因为构造函数的返回值默认为新创建的对象
    - 实践中为了从视觉上区分构造函数和普通函数，习惯将构造函数的首字母大写

- **实例成员**
  - 说明：通过构造函数创建的对象称为实例对象，实例对象中的属性和方法称为实例成员。

```
<script>
  // 构造函数
  function Person() {
    // 构造函数内部的 this 就是实例对象

    // 实例对象中动态添加属性
    this.name = '小明';
    // 实例对象动态添加方法（这种添加方法的方式不好，浪费空间，
    // 更好的做法是在构造函数的原型上加方法，让所有实例公用一个函数）
    this.sayHi = function () {
      console.log('大家好~');
    }
  }

  // 实例化，p1 是实例对象
  // p1 实际就是 构造函数内部的 this
```

```
  let p1 = new Person();

  console.log(p1);
  console.log(p1.name); // 访问实例属性
  p1.sayHi(); // 调用实例方法
</script>
```

- 总结：
  - 构造函数内部 this 实际上就是实例对象，为其动态添加的属性和方法即为实例成员
  - 为构造函数传入参数，动态创建结构相同但值不同的对象
  - 实例对象的 constructor 属性指向了构造函数（实际上constructor 在实例的原型中）
  - instanceof 用于检测实例对象对应的构造函数（底层向上兼容，例如数组的构造函数是Array也是Object，例如 [] instanceof Array 和 [] instanceof Object 结果都是true）
  - 构造函数创建的实例对象彼此独立互不影响

- **静态成员**
  - 说明：在 JavaScript 中底层函数本质上也是对象类型，因此允许直接为函数动态添加属性或方法，构造函数的属性和方法被称为静态成员。

```
<script>
  // 构造函数
  function Person(name, age) {
    // 省略实例成员
  }

  // 静态属性
  Person.eyes = 2;
  Person.arms = 2;
  // 静态方法
  Person.walk = function () {
    console.log('^_^人都会走路...');
    // this 指向 Person
    console.log(this.eyes);
  }
</script>
```

- 总结：
  - 静态成员指的是通过构造函数+点的方式添加到构造函数本身的属性和方法
  - 实例成员指的是直接写在构造方法里由this+点开头的属性和方法

    - 静态成员只能通过构造函数名点的方式调用，实例成员只能通过实例化的对象点的方式调用
    - 一般公共特征的属性或方法静态成员设置为静态成员
    - 静态成员方法中的 this 指向构造函数本身

- **一切皆对象**
  - 在 JavaScript 中最主要的数据类型有 6 种，分别是字符串、数字、布尔、undefined、函数、对象，常见的对象类型数据包括数组和普通对象。其中字符串、数字、布尔、undefined也被称为简单类型或基础类型，函数、对象也被称为引用类型。
  - 在 JavaScript 内置了一些构造函数，绝大部的数据处理都是基于这些构造函数实现的，JavaScript 基础阶段学习的 Date 就是内置的构造函数。

```
<script>
  // 实例化
    let date = new Date();

  // date 即为实例对象
  console.log(date);
</script>
```

  - 甚至字符串、数值、布尔、数组、普通对象也都有专门的构造函数，用于创建对应类型的数据。

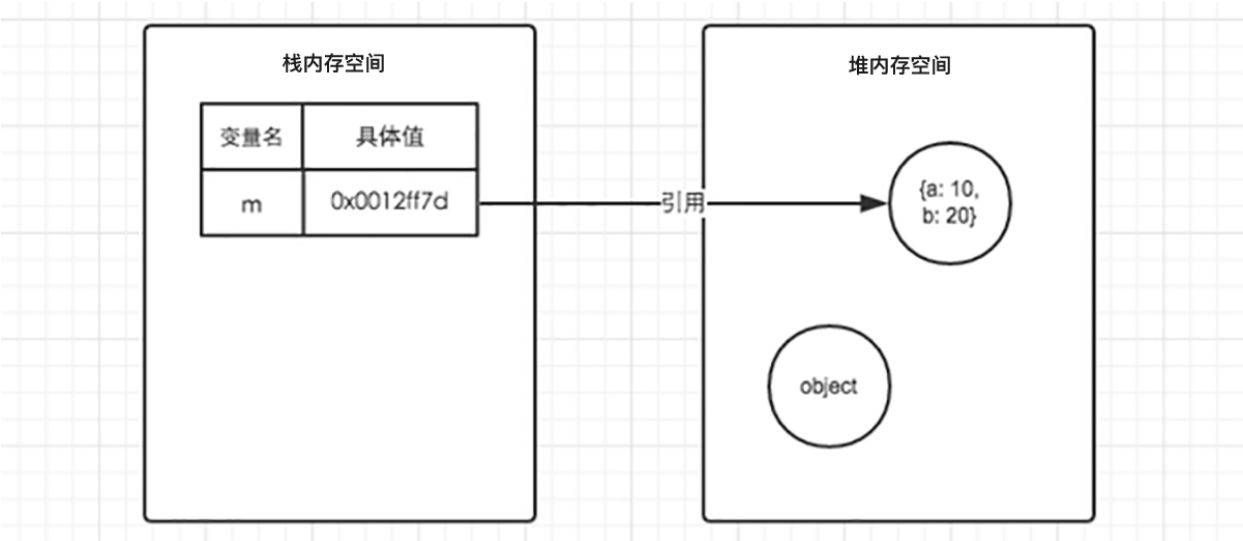
- **引用类型**
  - Object：是内置的构造函数，用于创建普通对象。
  - Array：是内置的构造函数，用于创建数组。
  - RegExp：内置的构造函数，用于创建正则表达式。

- **Object**
  - `Object` 是内置的构造函数，用于创建普通对象。

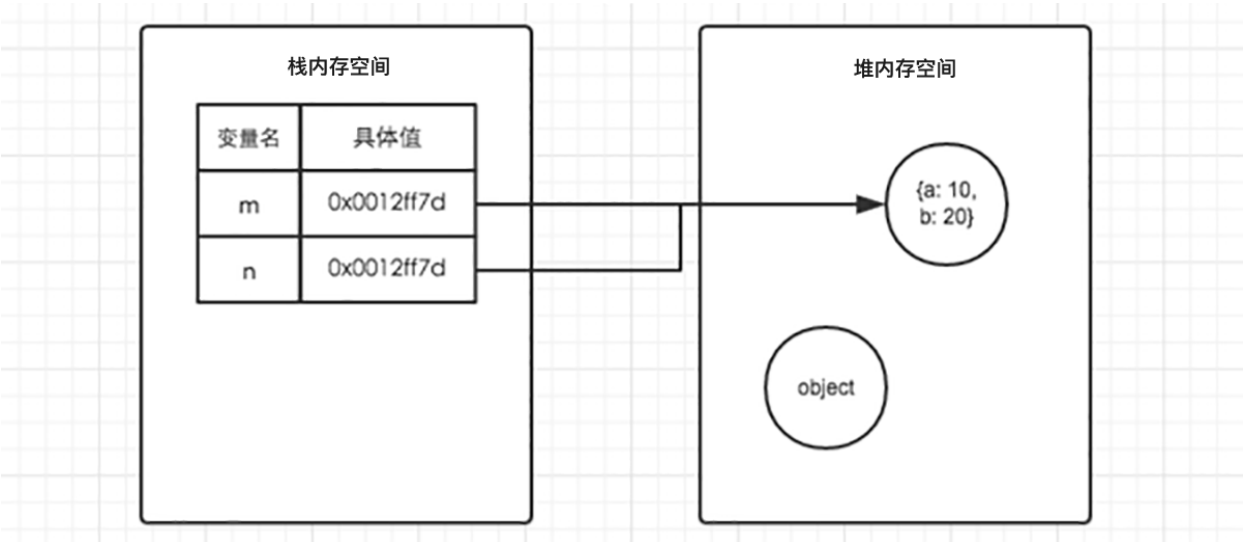
```
<script>
  // 通过构造函数创建普通对象
  let user = new Object({name: '小明', age: 15});

  // 这种方式声明的变量称为【字面量】
  let student = {name: '杜子腾', age: 21}

  // 对象语法简写
  let name = '小红';
```

```
  let people = {
    // 相当于 name: name
    name,
    // 相当于 walk: function () {}
    walk () {
      console.log('人都要走路...');
    }
  }

  console.log(student.constructor);
  console.log(user.constructor);
  console.log(student instanceof Object);
</script>
```

- 下图展示了普通对象在内存中的存储方式：普通对象数据保存在堆内存之中，栈内存中保存了普通对象在堆内存的地址。



- 普能对象在赋值时只是复制了栈内中的地址，而非堆内存中的数据，如下图所示：



- 普通对象赋值后，无论修改哪个变量另一个对象的数据值也会相当发生改变。

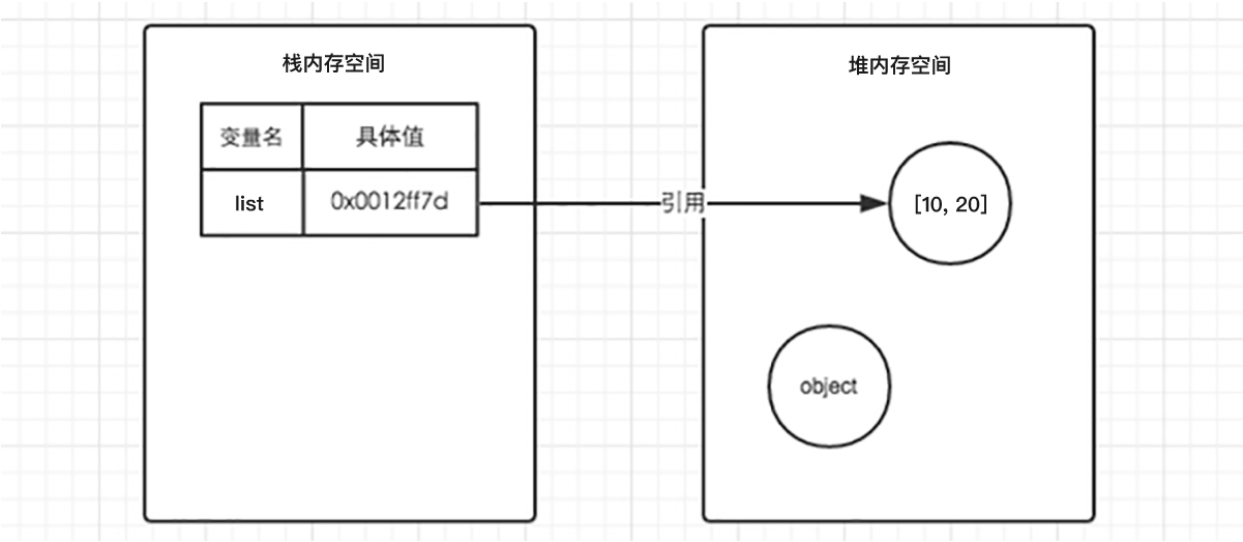
- 总结：

    - 推荐使用字面量方式声明对象，而不是 Object 构造函数
    - Object.assign 静态方法创建新的对象
    - Object.keys 静态方法获取对象中所有属性
    - Object.values 表态方法获取对象中所有属性值

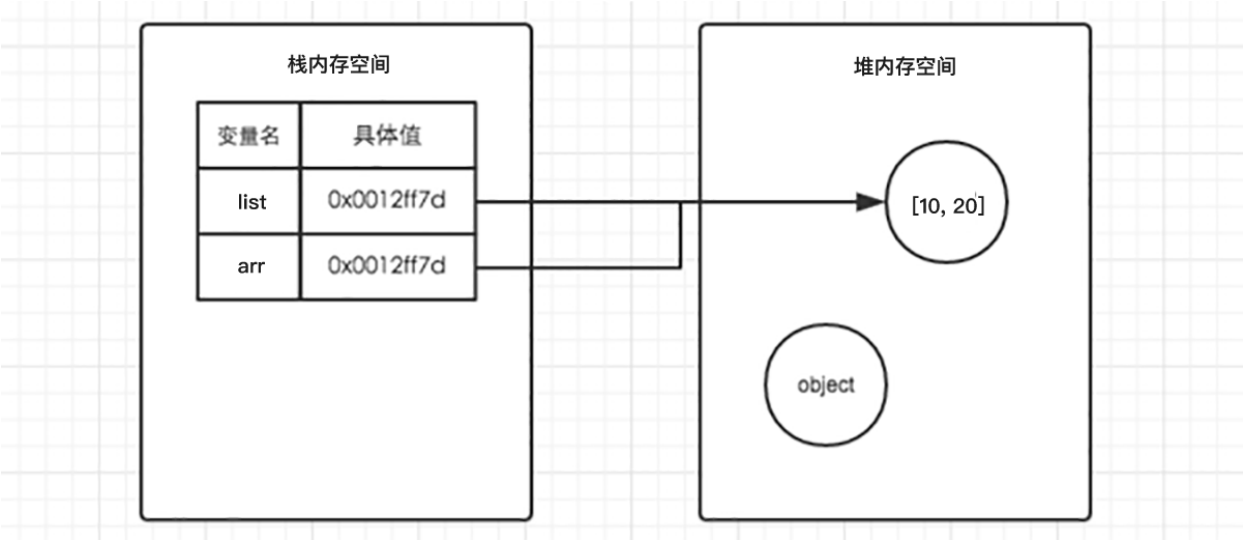
- **Array**
    - `Array` 是内置的构造函数，用于创建数组。

```
<script>
  // 构造函数创建数组
  let arr = new Array(5, 7, 8);

  // 字面量方式创建数组
  let list = ['html', 'css', 'javascript'];

  console.log(list.constructor);
  console.log(list instanceof Array);
</script>
```

    - 数组在内存中的存储方式与普通对象一样，如下图所示：



    - 数组在赋值时只是复制了栈内中的地址，而非堆内存中的数据，如下图所示：



    - 数组赋值后，无论修改哪个变量另一个对象的数据值也会相当发生改变。
  - 总结：
    - 推荐使用字面量方式声明数组，而不是 Array 构造函数
    - 实例方法 forEach 用于遍历数组，替代 for 循环
      - 实例方法 find 用于有功能的遍历数组：查找第一个符合返回条件的数据，返回数据
      - 实例方法 findIndex 用于有功能的遍历数组：查找第一个符合返回条件的数据，返回数据的下标
      - 实例方法 some 用于有功能的遍历数组：检测是否有符合返回条件的数据，返回布尔值
      - 实例方法 every 用于有功能的遍历数组：检测是否全部符合返回条件，返回布尔值
      - 实例方法 filter 用于有功能的遍历数组：查找所有符合返回条件的数据，返回数据生成的新数组
      - 实例方法 map 用于有功能的遍历数组：根据返回条件迭代所有数据，返回迭代后的数组
    - 实例方法 join 数组单元素拼接成了符串
    - 实例方法 concat 合并两个数组，生成新数组
    - 实例方法 sort 对原数组单元值排序
    - 实例方法 splice 删除或替换原数组单元
    - 实例方法 indexOf 检索某元素在数组中的第一个索引，没找到返回-1
      - 实例方法 lastIndexOf 检索某元素在数组中的最后一个索引，没找到返回-1
    - 实例方法 reverse 反转数组
    - 静态方法 from 伪数组转成数组
    - 静态方法 isArray 判断是否是数组

- **RegExp**
  - `RegExp` 内置的构造函数，用于创建正则表达式。

```
<script>
  // 构造函数创建正则
  let reg = new RegExp('\d', 'i');

  // 字面量方式创建正则
```

```
  // let reg = /(\d)/i;

  reg.exec('123');
</script>
```

- 总结：
  - 推荐使用字面量定义正则表达式，而不是 RegExp 构造函数
- 创建正则两种方法：
  - 字面量
  - 构造函数

- **堆与栈的区别**
  - 堆和栈是内存中的数据存储空间
  - 简单类型的数据保存在内存的栈空间中
  - 引用类型的数据保存在内存的堆空间中，栈内存中存取的是引用类型的地址

- **包装类型**
  - 在 JavaScript 中的字符串、数值、布尔具有对象的使用特征，如具有属性和方法，如下代码举例：

```
<script>
  // 字符串类型
  let str = 'hello world!';
    // 统计字符的长度（字符数量）
  console.log(str.length);

  // 数值类型
  let price = 12.345;
  // 保留两位小数
  price.toFixed(2);
</script>
```

  - 符串、数值、布尔之所以具有对象特征的原因是字符串、数值、布尔类型数据是 JavaScript 底层使用 Object 构造函数“包装”来的，被称为包装类型。

- **String**
  - `String` 是内置的构造函数，用于创建字符串。

```
<script>
```

```
  // 使用构造函数创建字符串
  let str = new String('hello world!');

  // 字面量创建字符串
  let str2 = '你好，世界！';

  // 检测是否属于同一个构造函数
  console.log(str.constructor === str2.constructor); // true
  console.log(str instanceof String); // false
</script>
```

- 总结：
  - 推荐使用字面量方式声明字符串，而不是 String 构造函数
  - 实例属性 length 用来获取字符串的度长
  - 实例方法 split 用来将字符串拆分成数组
  - 实例方法 toUpperCase 用于将字母转换成大写
  - 实例方法 toLowerCase 用于将字母转换成小写
  - 实例方法 slice 用于字符串截取，从第一个参数位置截取到第二个参数的位置，前闭后开，参数顺序颠倒了无法执行。
    - 实例方法 substring 用于字符串截取，从第一个参数位置截取到第二个参数的位置，前闭后开，参数顺序颠倒了自行纠错。
    - 实例方法 substr 用于字符串截取，从第一个参数位置截取第二个参数的长度。
  - 实例方法 indexOf 检测是否包含某字符，返回第一个目标字符的下标。
    - 实例方法 lastIndexOf 检测是否包含某字符，返回最后一个目标字符的下标。
  - 实例方法 startsWith 检测是否以某字符开头
  - 实例方法 endsWith 检测是否以某字符结尾
  - 实例方法 replace 用于替换字符串，支持正则匹配
  - 实例方法 trim 用于去除字符串开头和结尾处的空格
  - 注：String 也可以当做普通函数使用，这时它的作用是强制转换成字符串数据类型。

- **Number**
  - `Number` 是内置的构造函数，用于创建数值。

```
<script>
  // 使用构造函数创建数值
  let x = new Number('10');
  let y = new Number(5);

  // 字面量创建数值
```

```
  let z = 20;

  // 检测是否属于同一个构造函数
  console.log(x.constructor === z.constructor);
</script>
```

- 总结：
  - 推荐使用字面量方式声明数值，而不是 Number 构造函数
  - 实例方法 toFixed 用于设置保留小数位的长度
  - 注：Number 也可以当做普通函数使用，这时它的作用是强制转换成数值数据类型。

- **Boolean**
  - `Boolean` 是内置的构造函数，用于创建布尔值。

```
<script>
  // 使用构造函数创建布尔类型
  let locked = new Boolean('10');

  // 字面量创建布尔类型
  let flag = true;

  // 检测是否属于同一个构造函数
  console.log(locked.constructor === flag.constructor);
</script>
```

- 总结：
  - 推荐使用字面量方式声明布尔值，而不是 Boolean 构造函数
  - 注：Boolean 也可以当做普通函数使用，这时它的作用是强制转换成布尔类型数据，由其它数据类型转换成布尔类型的数据被称为真值（truly）或假值（falsly）。

- **公共方法**
  - 无论是引用类型或是包装包类型都包含两个公共的方法 `toString` 和 `valueOf`

```
<script>
  // 对象类型数据
  let user = {name: '小明', age: 18}
  // 数值类型
  let num = 12.345;
```

```
  // 字符串类型
  let str = 'hello world!';

  str.valueOf(); // 原始值
  user.toString(); // 表示该对象的字符串
</script>
```

- 总结：
  - valueOf 方法获取原始值，数据内部运算的基础，很少主动调用该方法
  - toString 方法以字符串形式表示对象